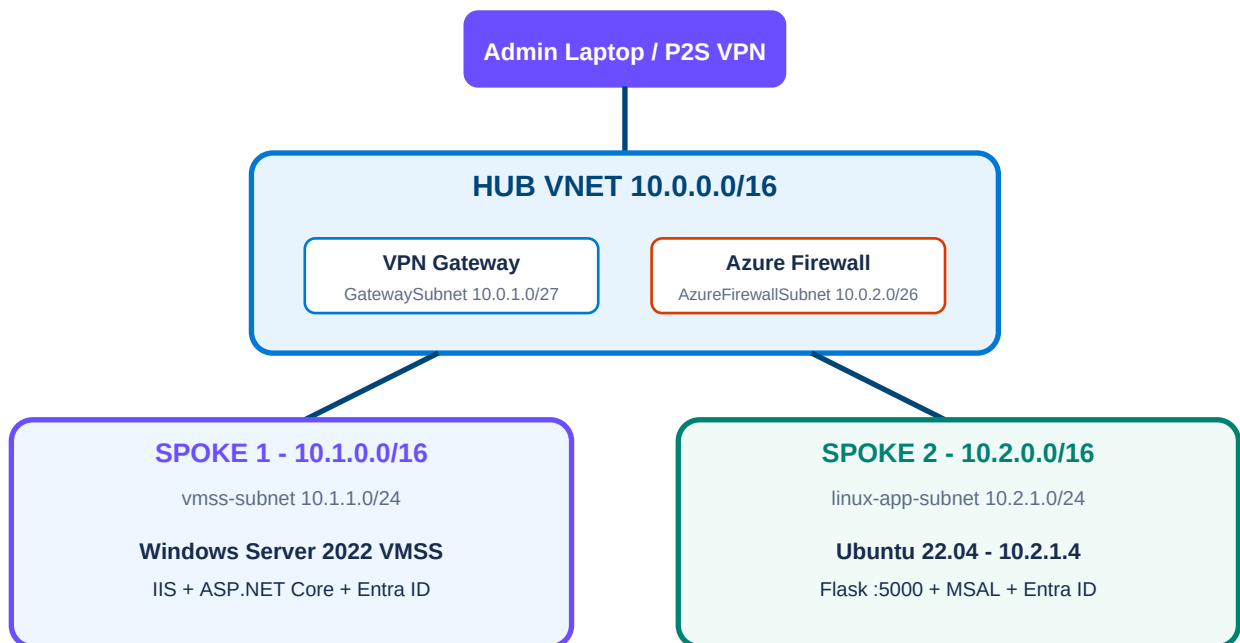


Azure Hub-Spoke Architecture with Microsoft Entra ID

Complete Deployment & Development Guide

Terraform Infrastructure-as-Code | P2S VPN | Azure Firewall | Windows VMSS | IIS | ASP.NET Core | Ubuntu | Flask | MSAL | Microsoft Entra ID | GitHub



Purpose: A reproducible, step-by-step guide that another engineer can follow to deploy the same private Azure lab safely. All credentials and tenant-specific values are represented as placeholders.

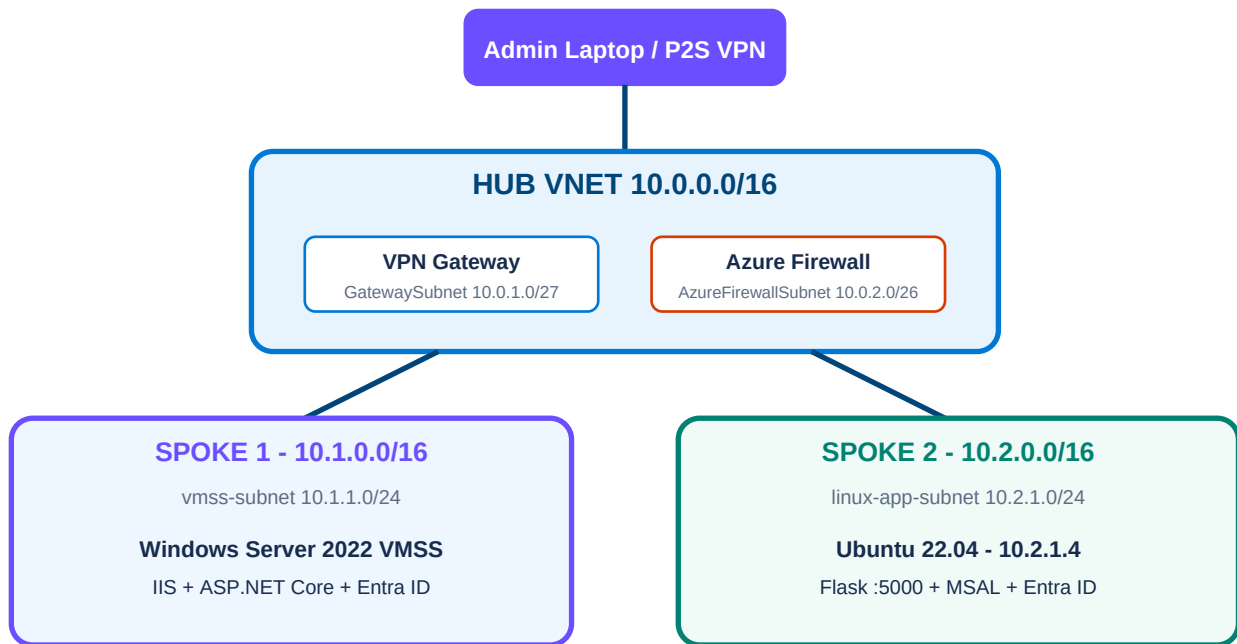
Security: Never publish passwords, client secrets, private SSH keys, PFX files, Terraform state, or VPN private certificates.

Contents

1. Architecture and deployment sequence
2. Prerequisites and repository clone
3. Terraform project structure
4. Providers, variables and safe input handling
5. Resource Group
6. Hub network
7. Spoke 1 network - Windows VMSS
8. Spoke 2 network - Linux Flask
9. Network Security Groups
10. Hub-Spoke peering and gateway transit
11. Azure Firewall
12. User-defined routing
13. Point-to-Site VPN Gateway
14. Windows VM Scale Set
15. VMSS autoscaling
16. Linux VM
17. Microsoft Entra app registrations with Terraform
18. Terraform outputs
19. P2S certificate preparation
20. Terraform deployment commands
21. Linux Flask + MSAL application deployment
22. Windows IIS + ASP.NET Core + Microsoft.Identity.Web
23. Entra login configuration and redirect URIs
24. Connectivity and validation tests
25. Troubleshooting
26. Git security and .gitignore
27. GitHub upload workflow
28. Clean deployment checklist

The infrastructure sections use separate Terraform files so the repository remains readable, reviewable and easy to troubleshoot.

1. Architecture and deployment sequence



- **Hub VNet:** 10.0.0.0/16
- **GatewaySubnet:** 10.0.1.0/27
- **AzureFirewallSubnet:** 10.0.2.0/26
- **Spoke 1:** 10.1.0.0/16 with VMSS subnet 10.1.1.0/24
- **Spoke 2:** 10.2.0.0/16 with Linux subnet 10.2.1.0/24
- **P2S client pool:** 172.16.100.0/24
- **Linux application:** 10.2.1.4:5000
- **Application identity:** separate Entra app registrations for Windows and Linux portals

Recommended deployment order

1	Authenticate to Azure Run Azure CLI login and select the intended subscription.
2	Deploy network foundation Resource group, hub, spokes, subnets, NSGs and peerings.
3	Deploy security and routing Azure Firewall, route tables and spoke routing.
4	Deploy private access VPN Gateway and certificate-based Point-to-Site VPN.

5

Deploy workloads

Windows VMSS and private Ubuntu VM.

6

Create Entra registrations

Create separate application registrations and redirect URIs.

7

Deploy applications

IIS/ASP.NET Core and Flask/MSAL.

8

Validate

Test VPN, spoke connectivity, application ports and Entra login.

9

Publish safely

Run secret checks, commit safe files and push to GitHub.

2. Prerequisites and repository clone

- Azure subscription and Microsoft Entra tenant
- Azure CLI
- Terraform
- Git
- PowerShell
- .NET SDK
- Python 3
- PuTTY (if using the localhost SSH tunnel for the Flask callback)

```
az --version
terraform version
git --version
dotnet --version
python --version
az login
az account show --output table
az account set --subscription "<SUBSCRIPTION-ID>"
```

Clone the repository

```
git clone https://github.com/rajan-iac/Azure-hub-spoke-entra-lab.git
cd Azure-hub-spoke-entra-lab
git status
```

Use your own subscription, tenant, credentials, certificates and keys. Do not reuse values from another deployment.

3. Terraform project structure

```
terraform/
├── providers.tf
├── variables.tf
├── terraform.tfvars.example
├── resource-group.tf
├── hub-network.tf
├── spoke1-network.tf
├── spoke2-network.tf
├── nsg.tf
├── peering.tf
├── firewall.tf
├── routing.tf
├── vpn-gateway.tf
├── windows-vmss.tf
├── autoscale.tf
├── linux-vm.tf
├── entra-id.tf
└── outputs.tf
```

Keep resource groups, networks, security, compute, identity and outputs in separate files. Terraform loads all .tf files in the directory as one configuration.

4. Providers, variables and safe input handling

providers.tf

```
terraform {
  required_version = ">= 1.9.0"
  required_providers {
```

```

    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 5.7"
    }
    azuread = {
      source = "hashicorp/azuread"
      version = "~> 3.9"
    }
  }
}
provider "azurerm" {
  features {}
}
provider "azuread" {}

```

variables.tf

```

variable "location" {
  description = "Azure deployment region"
  type        = string
  default     = "Central India"
}
variable "resource_group_name" {
  description = "Azure Resource Group"
  type        = string
  default     = "hub-spoke-lab-rg"
}
variable "vm_admin_username" {
  description = "Administrator username"
  type        = string
  default     = "azureadmin"
}
variable "vm_admin_password" {
  description = "Windows VMSS administrator password"
  type        = string
  sensitive   = true
}
variable "linux_ssh_public_key" {
  description = "SSH public key used for Linux VM"
  type        = string
  sensitive   = true
}
variable "vpn_root_certificate_data" {
  description = "Base64 X.509 public root certificate"
  type        = string
  sensitive   = true
}
variable "windows_redirect_uri" {
  description = "Exact HTTPS redirect URI used by the Windows Employee Portal"
  type        = string
  default     = "https://employeeportal.example.local/signin-oidc"
}

```

terraform.tfvars.example

```

location          = "Central India"
resource_group_name = "hub-spoke-lab-rg"
vm_admin_username  = "azureadmin"
vm_admin_password  = "REPLACE_WITH_YOUR_PASSWORD"
linux_ssh_public_key = "ssh-rsa REPLACE_WITH_YOUR_PUBLIC_KEY"
vpn_root_certificate_data = "REPLACE_WITH_BASE64_PUBLIC_CERTIFICATE"
windows_redirect_uri = "https://employeeportal.example.local/signin-oidc"

```

Recommended: provide the real VM password using an environment variable rather than storing it in a tracked file.

```

# Git Bash
export TF_VAR_vm_admin_password='YOUR-STRONG-PASSWORD'
# PowerShell
$env:TF_VAR_vm_admin_password = "YOUR-STRONG-PASSWORD"

```

5. Resource Group - resource-group.tf

```

resource "azurerm_resource_group" "hub_spoke" {

```

```

name      = var.resource_group_name
location  = var.location
tags = {
  Project      = "Azure-Hub-Spoke-Entra-Lab"
  Environment  = "Lab"
  ManagedBy    = "Terraform"
}
}

```

6. Hub network - hub-network.tf

```

resource "azurerm_virtual_network" "hub" {
  name            = "hub-spoke-lab-hub-vnet"
  address_space   = ["10.0.0.0/16"]
  location        = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  tags = {
    Network = "Hub"
  }
}
resource "azurerm_subnet" "gateway" {
  name            = "GatewaySubnet"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.hub.name
  address_prefixes = ["10.0.1.0/27"]
}
resource "azurerm_subnet" "firewall" {
  name            = "AzureFirewallSubnet"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.hub.name
  address_prefixes = ["10.0.2.0/26"]
}
}

```

Keep the reserved subnet names exactly as shown: **GatewaySubnet** and **AzureFirewallSubnet**.

7. Spoke 1 network - spoke1-network.tf

```

resource "azurerm_virtual_network" "spoke1" {
  name            = "spoke1_vnet"
  address_space   = ["10.1.0.0/16"]
  location        = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  tags = {
    Network = "Spoke1"
    Workload = "Windows-VMSS"
  }
}
resource "azurerm_subnet" "spoke1_vmss" {
  name            = "vmss-subnet"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.spoke1.name
  address_prefixes = ["10.1.1.0/24"]
}
}

```

8. Spoke 2 network - spoke2-network.tf

```

resource "azurerm_virtual_network" "spoke2" {
  name            = "spoke2_vnet"
  address_space   = ["10.2.0.0/16"]
  location        = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  tags = {
    Network = "Spoke2"
    Workload = "Linux-Flask"
  }
}
resource "azurerm_subnet" "spoke2_linux" {
  name            = "linux-app-subnet"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.spoke2.name
}

```

```

address_prefixes    = ["10.2.1.0/24"]
}

```

9. Network Security Groups - nsg.tf

```

resource "azurerm_network_security_group" "spoke1" {
  name                = "spoke1-nsg"
  location            = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
}
resource "azurerm_network_security_group" "spoke2" {
  name                = "spoke2-nsg"
  location            = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
}
resource "azurerm_network_security_rule" "spoke1_rdp_p2s" {
  name                = "Allow-RDP-P2S"
  priority            = 100
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_range = "3389"
  source_address_prefix = "172.16.100.0/24"
  destination_address_prefix = "10.1.1.0/24"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  network_security_group_name = azurerm_network_security_group.spoke1.name
}
resource "azurerm_network_security_rule" "spoke1_web_p2s" {
  name                = "Allow-Web-P2S"
  priority            = 110
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_ranges = ["80", "443"]
  source_address_prefix = "172.16.100.0/24"
  destination_address_prefix = "10.1.1.0/24"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  network_security_group_name = azurerm_network_security_group.spoke1.name
}
resource "azurerm_network_security_rule" "spoke1_web_spoke2" {
  name                = "Allow-Web-Spoke2"
  priority            = 120
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_ranges = ["80", "443"]
  source_address_prefix = "10.2.0.0/16"
  destination_address_prefix = "10.1.1.0/24"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  network_security_group_name = azurerm_network_security_group.spoke1.name
}
resource "azurerm_network_security_rule" "spoke2_ssh_p2s" {
  name                = "Allow-Ssh-P2S"
  priority            = 100
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_range = "22"
  source_address_prefix = "172.16.100.0/24"
  destination_address_prefix = "10.2.1.0/24"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  network_security_group_name = azurerm_network_security_group.spoke2.name
}
resource "azurerm_network_security_rule" "spoke2_flask_p2s" {
  name                = "Allow-Flask-P2S"
  priority            = 110
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_range = "5000"
  source_address_prefix = "172.16.100.0/24"
}

```

```

destination_address_prefix = "10.2.1.0/24"
resource_group_name        = azurerm_resource_group.hub_spoke.name
network_security_group_name = azurerm_network_security_group.spoke2.name
}
resource "azurerm_network_security_rule" "spoke2_flask_spoke1" {
  name                = "Allow-Flask-Spoke1"
  priority            = 120
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_range = "5000"
  source_address_prefix = "10.1.0.0/16"
  destination_address_prefix = "10.2.1.0/24"
  resource_group_name   = azurerm_resource_group.hub_spoke.name
  network_security_group_name = azurerm_network_security_group.spoke2.name
}
resource "azurerm_subnet_network_security_group_association" "spoke1" {
  subnet_id            = azurerm_subnet.spoke1_vmss.id
  network_security_group_id = azurerm_network_security_group.spoke1.id
}
resource "azurerm_subnet_network_security_group_association" "spoke2" {
  subnet_id            = azurerm_subnet.spoke2_linux.id
  network_security_group_id = azurerm_network_security_group.spoke2.id
}

```

These rules intentionally limit RDP, SSH and application ports to the P2S pool or the other spoke rather than exposing the workloads publicly.

10. Hub-Spoke peering and gateway transit - peering.tf

```

resource "azurerm_virtual_network_peering" "hub_to_spoke1" {
  name                = "hub-to-spoke1"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.hub.name
  remote_virtual_network_id = azurerm_virtual_network.spoke1.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
  allow_gateway_transit        = true
}
resource "azurerm_virtual_network_peering" "spoke1_to_hub" {
  name                = "spoke1-to-hub"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.spoke1.name
  remote_virtual_network_id = azurerm_virtual_network.hub.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
  use_remote_gateways          = true
  depends_on = [azurerm_virtual_network_gateway.vpn]
}
resource "azurerm_virtual_network_peering" "hub_to_spoke2" {
  name                = "hub-to-spoke2"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.hub.name
  remote_virtual_network_id = azurerm_virtual_network.spoke2.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
  allow_gateway_transit        = true
}
resource "azurerm_virtual_network_peering" "spoke2_to_hub" {
  name                = "spoke2-to-hub"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  virtual_network_name = azurerm_virtual_network.spoke2.name
  remote_virtual_network_id = azurerm_virtual_network.hub.id
  allow_virtual_network_access = true
  allow_forwarded_traffic      = true
  use_remote_gateways          = true
  depends_on = [azurerm_virtual_network_gateway.vpn]
}

```

Gateway transit allows the spoke VNets to use the VPN Gateway hosted in the Hub.

11. Azure Firewall - firewall.tf

```

resource "azurerm_public_ip" "firewall" {
  name           = "hub-spoke-lab-firewall-pip"
  location       = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  allocation_method = "Static"
  sku            = "Standard"
}
resource "azurerm_firewall" "hub" {
  name           = "hub_firewall"
  location       = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  sku_name       = "AZFW_VNet"
  sku_tier       = "Standard"
  ip_configuration {
    name           = "firewall-ipconfig"
    subnet_id      = azurerm_subnet.firewall.id
    public_ip_address_id = azurerm_public_ip.firewall.id
  }
}
resource "azurerm_firewall_network_rule_collection" "spoke1_to_spoke2" {
  name           = "Spoke1-To-Spoke2"
  azure_firewall_name = azurerm_firewall.hub.name
  resource_group_name = azurerm_resource_group.hub_spoke.name
  priority       = 100
  action         = "Allow"
  rule {
    name           = "Allow-Spoke1-To-Linux"
    source_addresses = ["10.1.0.0/16"]
    destination_addresses = ["10.2.0.0/16"]
    destination_ports = ["22", "5000"]
    protocols       = ["TCP"]
  }
}
resource "azurerm_firewall_network_rule_collection" "spoke2_to_spoke1" {
  name           = "Spoke2-To-Spoke1"
  azure_firewall_name = azurerm_firewall.hub.name
  resource_group_name = azurerm_resource_group.hub_spoke.name
  priority       = 110
  action         = "Allow"
  rule {
    name           = "Allow-Linux-To-Windows"
    source_addresses = ["10.2.0.0/16"]
    destination_addresses = ["10.1.0.0/16"]
    destination_ports = ["80", "443"]
    protocols       = ["TCP"]
  }
}

```

12. User-defined routing - routing.tf

```

resource "azurerm_route_table" "spoke1" {
  name           = "spoke1-route-table"
  location       = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
}
resource "azurerm_route" "spoke1_to_spoke2" {
  name           = "spoke1-to-spoke2-via-firewall"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  route_table_name = azurerm_route_table.spoke1.name
  address_prefix   = "10.2.0.0/16"
  next_hop_type    = "VirtualAppliance"
  next_hop_in_ip_address = azurerm_firewall.hub.ip_configuration[0].private_ip_address
}
resource "azurerm_subnet_route_table_association" "spoke1" {
  subnet_id      = azurerm_subnet.spoke1_vmss.id
  route_table_id = azurerm_route_table.spoke1.id
}
resource "azurerm_route_table" "spoke2" {
  name           = "spoke2-route-table"
  location       = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
}
resource "azurerm_route" "spoke2_to_spoke1" {
  name           = "spoke2-to-spoke1-via-firewall"
  resource_group_name = azurerm_resource_group.hub_spoke.name
}

```

```

route_table_name      = azurerm_route_table.spoke2.name
address_prefix        = "10.1.0.0/16"
next_hop_type         = "VirtualAppliance"
next_hop_in_ip_address = azurerm_firewall.hub.ip_configuration[0].private_ip_address
}
resource "azurerm_subnet_route_table_association" "spoke2" {
  subnet_id      = azurerm_subnet.spoke2_linux.id
  route_table_id = azurerm_route_table.spoke2.id
}

```

13. Point-to-Site VPN Gateway - vpn-gateway.tf

```

resource "azurerm_public_ip" "vpn_gateway" {
  name            = "hub-spoke-lab-vpn-gateway-pip"
  location        = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  allocation_method = "Static"
  sku             = "Standard"
}
resource "azurerm_virtual_network_gateway" "vpn" {
  name            = "hub-spoke-lab-vpn-gateway"
  location        = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  type            = "Vpn"
  vpn_type        = "RouteBased"
  active_active    = false
  enable_bgp       = false
  sku              = "VpnGw1AZ"
  ip_configuration {
    name                = "vpn-gateway-ipconfig"
    public_ip_address_id = azurerm_public_ip.vpn_gateway.id
    private_ip_address_allocation = "Dynamic"
    subnet_id           = azurerm_subnet.gateway.id
  }
  vpn_client_configuration {
    address_space = ["172.16.100.0/24"]
    vpn_client_protocols = ["IkeV2"]
    root_certificate {
      name            = "HubSpokeRootCert"
      public_cert_data = var.vpn_root_certificate_data
    }
  }
}

```

The VPN certificate authenticates the administrator's laptop to the private Azure network. It is separate from Entra authentication used by the applications.

14. Windows VM Scale Set - windows-vmss.tf

```

resource "azurerm_windows_virtual_machine_scale_set" "web_vmss" {
  name            = "hub-spoke-lab-web-vmss"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  location        = azurerm_resource_group.hub_spoke.location
  sku             = "Standard_B2s"
  instances        = 1
  computer_name_prefix = "webvm"
  admin_username    = var.vm_admin_username
  admin_password     = var.vm_admin_password
  upgrade_mode       = "Manual"
  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
  os_disk {
    storage_account_type = "StandardSSD_LRS"
    caching               = "ReadWrite"
  }
  network_interface {
    name      = "web-vmss-nic"
    primary   = true
    ip_configuration {

```

```

    name      = "internal"
    primary   = true
    subnet_id = azurerm_subnet.spoke1_vmss.id
  }
}
lifecycle {
  ignore_changes = [instances]
}
}
}

```

No public IP is configured. RDP and web access are intended to occur over the private P2S path.

15. VMSS autoscaling - autoscale.tf

```

resource "azurerm_monitor_autoscale_setting" "vmss" {
  name                        = "hub-spoke-lab-vmss-autoscale"
  resource_group_name        = azurerm_resource_group.hub_spoke.name
  location                   = azurerm_resource_group.hub_spoke.location
  target_resource_id         = azurerm_windows_virtual_machine_scale_set.web_vmss.id
  profile {
    name = "default"
    capacity {
      default = 1
      minimum = 1
      maximum = 3
    }
    rule {
      metric_trigger {
        metric_name      = "Percentage CPU"
        metric_resource_id = azurerm_windows_virtual_machine_scale_set.web_vmss.id
        time_grain        = "PT1M"
        statistic          = "Average"
        time_window       = "PT5M"
        time_aggregation  = "Average"
        operator           = "GreaterThan"
        threshold          = 70
      }
      scale_action {
        direction = "Increase"
        type      = "ChangeCount"
        value     = "1"
        cooldown  = "PT5M"
      }
    }
  }
  rule {
    metric_trigger {
      metric_name      = "Percentage CPU"
      metric_resource_id = azurerm_windows_virtual_machine_scale_set.web_vmss.id
      time_grain        = "PT1M"
      statistic          = "Average"
      time_window       = "PT5M"
      time_aggregation  = "Average"
      operator           = "LessThan"
      threshold          = 30
    }
    scale_action {
      direction = "Decrease"
      type      = "ChangeCount"
      value     = "1"
      cooldown  = "PT5M"
    }
  }
}
}
}

```

16. Linux VM - linux-vm.tf

```

resource "azurerm_network_interface" "linux" {
  name                = "linuxapp01-nic"
  location             = azurerm_resource_group.hub_spoke.location
  resource_group_name = azurerm_resource_group.hub_spoke.name
  ip_configuration {
    name = "internal"
  }
}

```

```

    subnet_id                = azurerm_subnet.spoke2_linux.id
    private_ip_address_allocation = "Static"
    private_ip_address        = "10.2.1.4"
  }
}
resource "azurerm_linux_virtual_machine" "linux_app" {
  name                = "hub-spoke-lab-linux-app"
  computer_name       = "linuxapp01"
  resource_group_name = azurerm_resource_group.hub_spoke.name
  location            = azurerm_resource_group.hub_spoke.location
  size               = "Standard_B2s"
  admin_username      = var.vm_admin_username
  disable_password_authentication = true
  network_interface_ids = [
    azurerm_network_interface.linux.id
  ]
  admin_ssh_key {
    username   = var.vm_admin_username
    public_key = var.linux_ssh_public_key
  }
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "StandardSSD_LRS"
  }
  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}

```

17. Microsoft Entra app registrations - entra-id.tf

Application authentication flow



OpenID Connect / authorization code

MFA is enforced by Entra policy when applicable

Create separate app registrations for the Windows and Linux portals. This keeps callback URIs and application credentials isolated.

```

resource "azuread_application_registration" "windows_portal" {
  display_name      = "Windows-Employee-Portal"
  sign_in_audience = "AzureADMyOrg"
  description       = "Entra authentication for Windows IIS ASP.NET Core Employee Portal"
}
resource "azuread_application_redirect_uris" "windows_portal" {
  application_id = azuread_application_registration.windows_portal.id
  type          = "Web"
  redirect_uris = [
    var.windows_redirect_uri
  ]
}
resource "azuread_application_registration" "linux_portal" {
  display_name      = "Linux-Flask-Employee-Portal"
  sign_in_audience = "AzureADMyOrg"
  description       = "Entra authentication for Linux Flask Employee Portal"
}
resource "azuread_application_redirect_uris" "linux_portal" {
  application_id = azuread_application_registration.linux_portal.id
}

```

```

type      = "Web"
redirect_uris = [
  "http://localhost:5000/getAToken"
]
}

```

Client secret: For this learning/public repository, create the Linux client secret outside the public Terraform example and store it only in the VM's protected .env file. Never commit the secret value.

18. Terraform outputs - outputs.tf

```

data "azuread_client_config" "current" {}
output "resource_group_name" {
  value = azurerm_resource_group.hub_spoke.name
}
output "hub_vnet_name" {
  value = azurerm_virtual_network.hub.name
}
output "hub_vnet_id" {
  value = azurerm_virtual_network.hub.id
}
output "spoke1_vnet_name" {
  value = azurerm_virtual_network.spoke1.name
}
output "spoke2_vnet_name" {
  value = azurerm_virtual_network.spoke2.name
}
output "linux_app_private_ip" {
  value = azurerm_network_interface.linux.private_ip_address
}
output "azure_firewall_private_ip" {
  value = azurerm_firewall.hub.ip_configuration[0].private_ip_address
}
output "vpn_gateway_name" {
  value = azurerm_virtual_network_gateway.vpn.name
}
output "windows_entra_client_id" {
  value = azuread_application_registration.windows_portal.client_id
}
output "linux_entra_client_id" {
  value = azuread_application_registration.linux_portal.client_id
}
output "tenant_id" {
  value = data.azuread_client_config.current.tenant_id
}

```

19. P2S certificate preparation

Create root certificate - Windows PowerShell

```

$rootCert = New-SelfSignedCertificate `
  -Type Custom `
  -KeySpec Signature `
  -Subject "CN=HubSpokeRootCert" `
  -KeyExportPolicy Exportable `
  -HashAlgorithm sha256 `
  -KeyLength 2048 `
  -CertStoreLocation "Cert:\CurrentUser\My" `
  -KeyUsageProperty Sign `
  -KeyUsage CertSign

```

Create client certificate

```

$clientCert = New-SelfSignedCertificate `
  -Type Custom `
  -DnsName "HubSpokeClientCert" `
  -KeySpec Signature `
  -Subject "CN=HubSpokeClientCert" `
  -KeyExportPolicy Exportable `
  -HashAlgorithm sha256 `
  -KeyLength 2048 `

```

```
-CertStoreLocation "Cert:\CurrentUser\My" `
-Signer $rootCert
```

Terraform receives only the public root certificate body. The private PFX/client key must remain private and must not be committed.

20. Terraform deployment commands

1

Authenticate

Run the command shown below.

```
az login
az account show --output table
```

2

Enter Terraform directory

Run the command shown below.

```
cd terraform
```

3

Initialize

Run the command shown below.

```
terraform init
```

4

Format

Run the command shown below.

```
terraform fmt -recursive
```

5

Validate

Run the command shown below.

```
terraform validate
```

6

Plan

Run the command shown below.

```
terraform plan -out=tfplan
```

7

Apply

Run the command shown below.

```
terraform apply tfplan
```

8

Review outputs

Run the command shown below.

```
terraform output
```

Azure CLI validation

```
az resource list --resource-group hub-spoke-lab-rg --output table
az network vnet peering list --resource-group hub-spoke-lab-rg --vnet-name hub-spoke-lab-hub-
vnet --output table
az vm list-ip-addresses --resource-group hub-spoke-lab-rg --name hub-spoke-lab-linux-app
--output table
```

21. Linux Flask + MSAL application deployment

Connect after P2S VPN is established

```
ssh azureadmin@10.2.1.4
```

Install runtime

```
sudo apt update
sudo apt upgrade -y
sudo apt install python3 python3-pip python3-venv -y
sudo mkdir -p /opt/flaskapp
sudo chown $USER:$USER /opt/flaskapp
cd /opt/flaskapp
python3 -m venv .venv
source .venv/bin/activate
pip install flask msal python-dotenv requests
pip freeze > requirements.txt
```

.env

```
TENANT_ID=<YOUR-TENANT-ID>
CLIENT_ID=<YOUR-LINUX-APP-CLIENT-ID>
CLIENT_SECRET=<YOUR-ROTATED-SECRET>
REDIRECT_PATH=/getAToken
```

```
chmod 600 /opt/flaskapp/.env
```

Minimal MSAL configuration concept

```
import os
from dotenv import load_dotenv
load_dotenv()
TENANT_ID = os.getenv("TENANT_ID")
CLIENT_ID = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
AUTHORITY = f"https://login.microsoftonline.com/{TENANT_ID}"
REDIRECT_PATH = "/getAToken"
# Flask application must listen on the private VM interface.
app.run(host="0.0.0.0", port=5000)
```

systemd service

```
[Unit]
Description=Azure Hub Spoke Flask Application
After=network.target
[Service]
User=root
WorkingDirectory=/opt/flaskapp
ExecStart=/opt/flaskapp/.venv/bin/python /opt/flaskapp/app.py
Restart=always
[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable flaskapp
sudo systemctl restart flaskapp
sudo systemctl status flaskapp --no-pager
curl http://localhost:5000
```

PuTTY local tunnel for localhost callback

- **Source port:** 5000
- **Destination:** 127.0.0.1:5000
- **Type:** Local
- **Browser:** http://localhost:5000

- **Entra redirect URI:** `http://localhost:5000/getAToken`

22. Windows IIS + ASP.NET Core + Microsoft.Identity.Web

Install IIS

```
Install-WindowsFeature Web-Server -IncludeManagementTools
Get-WindowsFeature Web-Server
```

Install the appropriate ASP.NET Core Hosting Bundle on the VMSS instance before hosting the ASP.NET Core application in IIS.

Build and publish Employee Portal

```
cd C:\WebApps\EmployeePortal
dotnet restore
dotnet build
dotnet list package
dotnet publish -c Release -o C:\WebApps\EmployeePortal-Publish
```

Entra configuration - appsettings example

```
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<YOUR-TENANT-ID>",
    "ClientId": "<YOUR-WINDOWS-APP-CLIENT-ID>",
    "CallbackPath": "/signin-oidc"
  }
}
```

Program.cs authentication

```
builder.Services
    .AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApp(
        builder.Configuration.GetSection("AzureAd"));
```

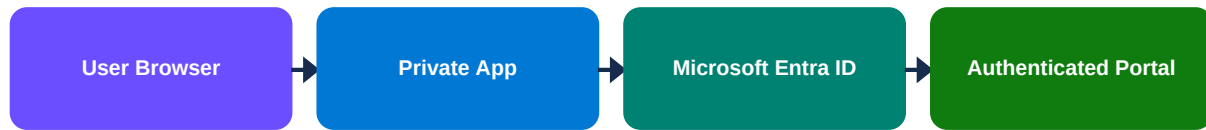
Deploy to IIS

```
Import-Module WebAdministration
Stop-WebAppPool -Name "EmployeePortal"
Remove-Item "C:\inetpub\EmployeePortal\*" -Recurse -Force
Copy-Item `
    "C:\WebApps\EmployeePortal-Publish\*" `
    "C:\inetpub\EmployeePortal\" `
    -Recurse -Force
Start-WebAppPool -Name "EmployeePortal"
iisreset
```

Stop the application pool before replacing binaries to avoid DLL file-locking problems.

23. Entra login configuration and redirect URIs

Application authentication flow



OpenID Connect / authorization code

MFA is enforced by Entra policy when applicable

Windows portal

- Create or use the **Windows-Employee-Portal** app registration.
- Platform type: **Web**.
- Callback path: **/signin-oidc**.
- Register the exact HTTPS URI that users browse to.
- Tenant ID and Client ID go into application configuration.

Linux Flask portal

- Create or use the **Linux-Flask-Employee-Portal** app registration.
- Platform type: **Web**.
- Redirect URI for this lab: **http://localhost:5000/getAToken**.
- Create a client secret and store the value only in the protected Linux .env file.

MFA: MFA is controlled by Microsoft Entra policy such as Conditional Access/security configuration. It is not implemented by hardcoding a second-factor mechanism in the application.

24. Connectivity and validation tests

P2S VPN

```
ipconfig
```

The VPN adapter should receive an address from 172.16.100.0/24.

Laptop -> Linux

```
ssh azureadmin@10.2.1.4
curl http://10.2.1.4:5000
```

Windows VMSS -> Linux Flask

```
Test-NetConnection 10.2.1.4 -Port 5000
Invoke-WebRequest http://10.2.1.4:5000
```

Linux -> Windows portal

```
curl -I http://<WINDOWS-VMSS-PRIVATE-IP>
curl -I https://<EMPLOYEE-PORTAL-HOSTNAME>
```

Expected validation matrix

Component	Expected result
Hub/Spoke peerings	Connected
P2S VPN	Connected; client receives 172.16.100.x
Windows VMSS	Private IP only; RDP over P2S
IIS / ASP.NET	Portal responds privately
Linux VM	10.2.1.4
Flask	TCP/5000 responds
Windows Entra login	Redirects to Entra and returns to /signin-oidc
Linux Entra login	Redirects to Entra and returns to /getAToken
Spoke 1 <-> Spoke 2	Allowed application traffic succeeds

25. Troubleshooting

AADSTS50011 - redirect URI mismatch

The URI registered under Entra App Registration > Authentication must exactly match the URI used by the application, including scheme, host, port and path.

Wrong Client ID or Tenant ID

- Check Entra ID > App registrations > Application > Overview.
- Copy the Application (client) ID into the correct application configuration.
- Use the Directory (tenant) ID for the intended tenant.

Linux login callback fails

- Confirm PuTTY local tunnel source 5000 -> destination 127.0.0.1:5000.
- Confirm Flask is listening on port 5000.
- Confirm http://localhost:5000/getAToken is registered.
- Confirm the SSH/P2S session remains connected while testing.

Terraform wants to recreate existing resources

Do not casually rename Terraform resource labels in an existing state. For a fresh clone the clean labels in this guide are fine. For an already deployed environment, preserve existing labels or deliberately migrate state with moved blocks/state operations.

26. Git security and .gitignore

```
# Terraform
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
tfplan
terraform.tfvars
*.auto.tfvars
```

```
# Keep .terraform.lock.hcl committed for reproducibility.
# SSH
*.ppk
*.pem
*.key
privatekey-linux
privatekey-linux.ppk
publickey-linux
# Certificates / VPN
*.pfx
*.p12
vpnclientconfiguration/
vpnclientconfiguration.zip
# Entra / application secrets
.env
.env.*
!.env.example
appsettings.Production.json
secrets.json
# .NET
bin/
obj/
publish/
# Python
.venv/
venv/
__pycache__/
*.pyc
# OS
Thumbs.db
.DS_Store
```

Pre-commit checks

```
git check-ignore -v terraform/terraform.tfvars
git check-ignore -v privatekey-linux.ppk
git check-ignore -v vpnclientconfiguration.zip
grep -RniE 'password|secret|client_secret|private_key|BEGIN.*PRIVATE' . --exclude-dir=.git
--exclude-dir=.terraform
```

If a real secret is found, rotate it before publishing. Adding a file to .gitignore after it was committed does not remove the secret from Git history.

27. GitHub upload workflow

Repository: **rajan-iac/Azure-hub-spoke-entra-lab**

1

Initialize local repository

Use the following Git command(s).

```
git init
git branch -M main
```

2

Review safe files

Use the following Git command(s).

```
git status
```

3

Stage

Use the following Git command(s).

```
git add .
```

4

Review staged content again

Use the following Git command(s).

```
git status
```

5**Commit**

Use the following Git command(s).

```
git commit -m "Add complete Azure Hub-Spoke Terraform deployment"
```

6**Set remote**

Use the following Git command(s).

```
git remote add origin https://github.com/rajan-iac/Azure-hub-spoke-entra-lab.git
```

7**Verify remote**

Use the following Git command(s).

```
git remote -v
```

8**Push**

Use the following Git command(s).

```
git push -u origin main
```

If origin already exists

```
git remote set-url origin https://github.com/rajan-iac/Azure-hub-spoke-entra-lab.git
```

For later updates

```
git status
git add .
git commit -m "Update deployment documentation"
git pull --rebase origin main
git push origin main
```

For a brand-new remote repository, avoid creating a separate README/.gitignore/license commit on GitHub before the first push if you already have local history. That prevents unrelated-history merge conflicts.

28. Clean deployment checklist

Status	Validation item
■	Azure CLI authenticated to correct subscription
■	Terraform initialized and validated
■	Hub VNet and reserved subnets deployed
■	Spoke 1 and Spoke 2 deployed
■	NSGs associated and required ports restricted
■	Hub-Spoke peerings Connected
■	Azure Firewall and UDRs deployed
■	VPN Gateway deployed and P2S client connects
■	Windows VMSS reachable privately

Status	Validation item
■	Linux VM reachable at 10.2.1.4
■	Flask responds on TCP/5000
■	Windows Entra app registration configured
■	Linux Entra app registration configured
■	Redirect URIs exactly match application callbacks
■	Windows Entra login validated
■	Linux Entra login validated
■	Spoke-to-Spoke application traffic validated
■	No secrets, private keys, state or PFX files staged in Git
■	GitHub repository pushed successfully

Final architecture principle: P2S certificate authentication provides private network access to Azure. Microsoft Entra ID authenticates users to the applications. These are two separate security layers and should remain documented separately.

End of Development Guide